

Policy Conflict Internals in Real Agentic Contexts

Opener

This work follows up on our recent blog post documenting our collaboration with DX Terminal to use mechanistic interpretability in real world financial contexts. In part 1, we described our research probing for, and discovering, the early signs of interpretable “Market Perception” geometries in LLMs when given real market data.

Part 2 of our work examined a common problem DXRG saw in their agents: strange behaviors when policies collide. Users often ask their agents to execute on strategies they create that directly conflict with the vault settings they inputted when configuring the initial setup.

Our thesis was that we’d be able to find interpretable probe directions associated with “conflict” that might allow us to detect difficult-to-spot policy-conflict scenarios in the prompts.

We used a semi-synthetic pipeline to discover 3 directions associated with distinct “conflict families” that shared geometric structure with each other, and have achieved early results that show the directions are active in real data in the cases they were trained to fire on.

Problem

A trading agent sees system rules, user strategies, active vault settings, portfolio state, market data, and prior decisions. Those sources are not always mutually consistent.

TODO: Add one concrete DX Terminal example in plain language: strategy says buy aggressively while settings cap size; strategy says sell all while settings imply partial exits; complaint refers to a strategy no longer active.

Why This Matters

One of our primary goals at Concordance is to bring frontier interpretability techniques and tools into the hands of all agent builders to enable them to learn more deeply about the LLMs they are building on every day. Given the field is still immature, and most of the findings from big labs have been directed towards “bigger” problems of safety and alignment, there is little work being done that can directly transfer to real problems on the ground.

The “policy conflict” problem was a real issue that the DXRG team noticed at various points both before and during their DX Terminal experiment, and we felt that it was the right ground for interpretability experiments. The central question became: can we detect policy conflicts in real prompts with a simple set of linear probes?

We selected this question because there was real data to prove it matters, and is scoped enough to sharpen our toolkit with possible downstream applications in monitoring, auditing, prompt optimization/UX redesigns, and deeper learnings that inform DXRG’s agent development.

Synthetic Abstraction

Real world data can be incredibly messy, but to ask mechanistic questions, one needs a clean dataset to amplify potentially active features before bringing learnings back into real settings.

In our case, we wanted to examine specific conflicts between user-configured **strategies** and vault-configured **settings**, so we created a synthetic dataset of 1100 rows to make the read stronger.

Synthetic Prompt Structure:

```
[system]
Role: trading agent.
Core rule: each prompt contains STRATEGY and ACTIVE SETTINGS.
Priority rule: ACTIVE SETTINGS are binding execution constraints.
Decision order:
```

1. decide whether ACTIVE SETTINGS permit entry
2. choose asset according to the allowed risk/diversification posture
3. choose size according to ACTIVE SETTINGS

Output format: strict JSON only.

[user]

TASK

Choose exactly one action for this tick.

STRATEGY

A compact preference statement, e.g.:

- prefer high-conviction trades
- prefer large/small size
- prefer concentrated/diversified exposure

Strategy applies only within ACTIVE SETTINGS.

ACTIVE SETTINGS

Slider-like constraints:

- Trading Activity
- Trade Size
- Risk Preference
- Holding Style
- Diversification

PORTFOLIO

Small controlled portfolio state.

MARKET

Four synthetic assets:

- ALPHA
- BETA
- DELTA
- GAMMA

Each has controlled evidence/risk/diversification language.

Using this structure, we can create a synthetic dataset that creates both aligned and conflicted rows between three setting types: trade_size, risk_preference, and diversification_preference. For example, to create a conflict row for trade_size, we could set the Trade Size slider to 5 (highest size), and then add a strategy like “Never trade more than 10% of portfolio” while keeping all other things constant. Our decision to isolate conflicts into three families came from an earlier unsupervised discovery phase that showed there might exist different conflict resolution circuits depending on the **type** of conflict in the prompt.

To avoid lexical confounds, we came up with strategy and other contextual information variation and split the data for strict holdouts to ensure there is minimal leakage between train and test sets.

TODO: Add lexical variation examples here.

Families Tested

- trade_size: buy small vs large; output size/action axis.
- risk_preference: asset selection by allowed risk posture.
- diversification_preference: concentration vs broadening; portfolio-conditioned.

Source: Generators: phase_09/scripts/build_phase_09_dataset.py, phase_10/scripts/build_phase_10_dataset.py, phase_12/scripts/build_phase_12_dataset.py.

Synthetic Probe Results Intro

After a few iterations on the synthetic prompt structure and confound isolation, we achieved the following results:

Synthetic Probe Table

Family	*Standard probe results*	*Strict holdout*
trade_size	XOR 0.9948 / 1.0000; strategy 1.0000 / 1.0000; settings 0.9948 / 1.0000	0.990 / 1.000 at L40
risk_preference	XOR 0.9635 / 0.9766; strategy 0.9844 / 0.9937; settings 0.9740 / 0.9839	0.8854 / 0.9119

diversification_preference	behavior aligned 1.0000, conflict 0.8542; XOR 0.9896 / 0.9995; strategy 1.0000 / 1.0000; settings 0.9792 / 0.9957	0.8333 / 0.8819
----------------------------	---	-----------------

Source: phase_12/reports/PROMPT_CONFLICT_FAMILY_CHECKPOINT_2026_04_16.typ

Figures

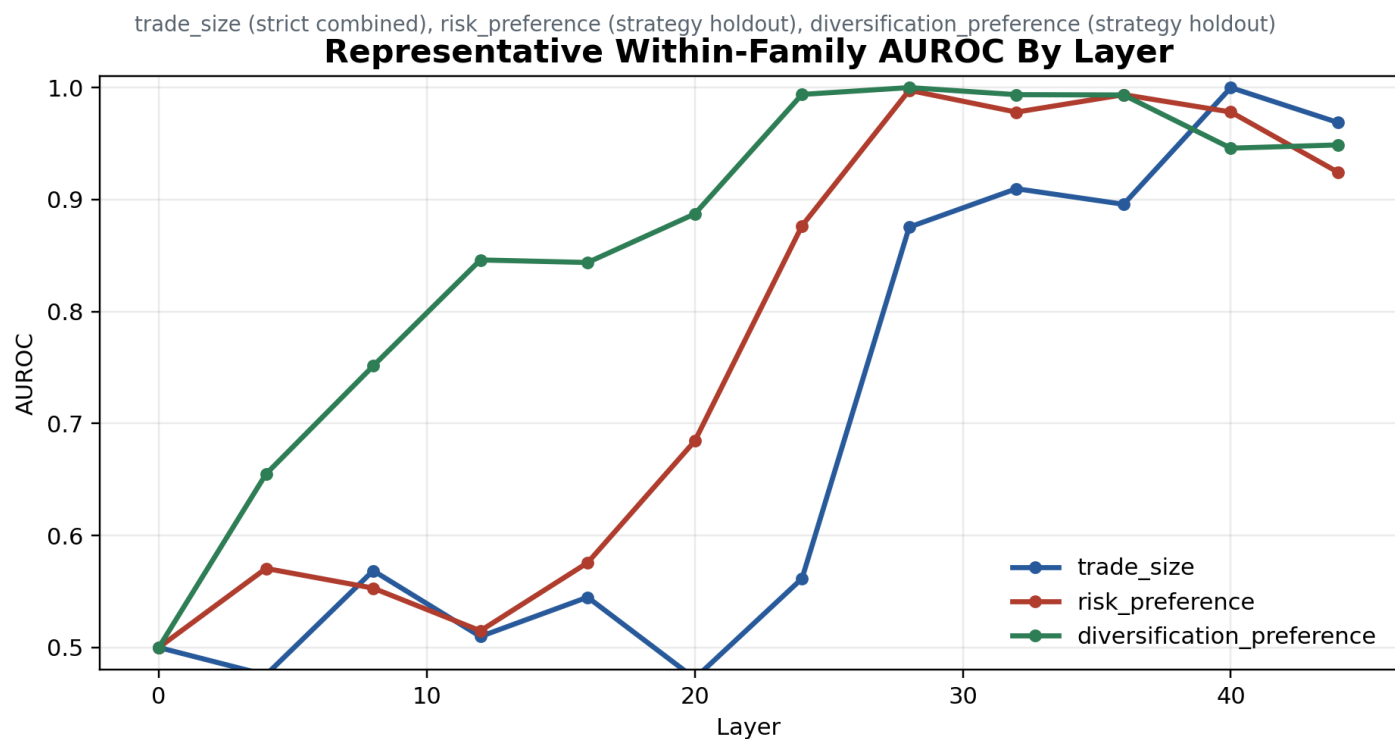


Figure 1: Representative within-family AUROC curves.

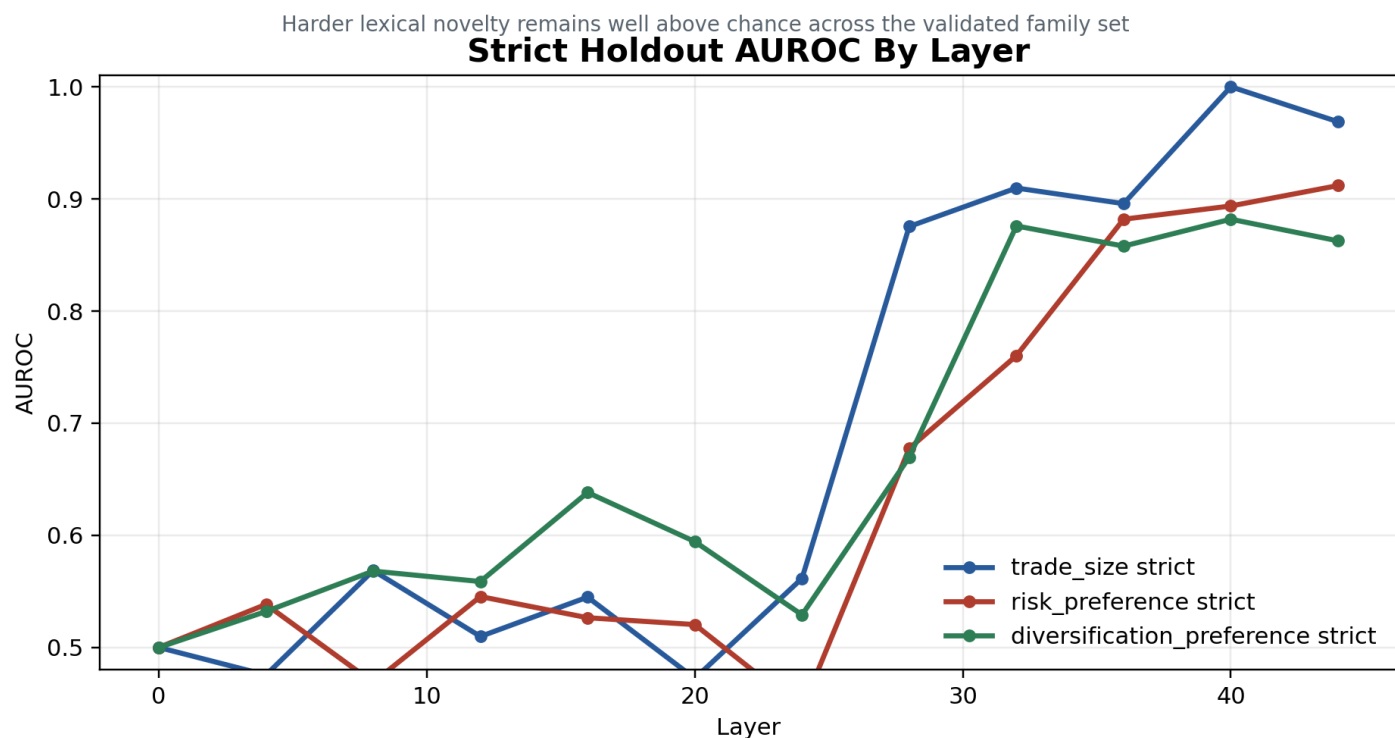


Figure 2: Strict lexical-holdout AUROC curves.

Synthetic Results Takeaway

We found that while the three aforementioned families had noticeably distinct geometry (see cosine similarities/PCA projections), there was a significant shared space when looking at the shared_mean across all trained probe vectors that hinted there is enough structure to continue in our tests.

- clean synthetic policy-conflict directions are strongly readable
- validated synthetic families:
 - trade_size
 - risk_preference
 - diversification_preference
- families share meaningful geometry
- not collinear

L36 Same-Capture Geometry

Pair	*Cosine*
risk_preference vs trade_size	0.6449
diversification_preference vs risk_preference	0.4684
diversification_preference vs trade_size	0.4883

Source: phase_12/reports/three_family_visuals/summary.json

Geometry Figures

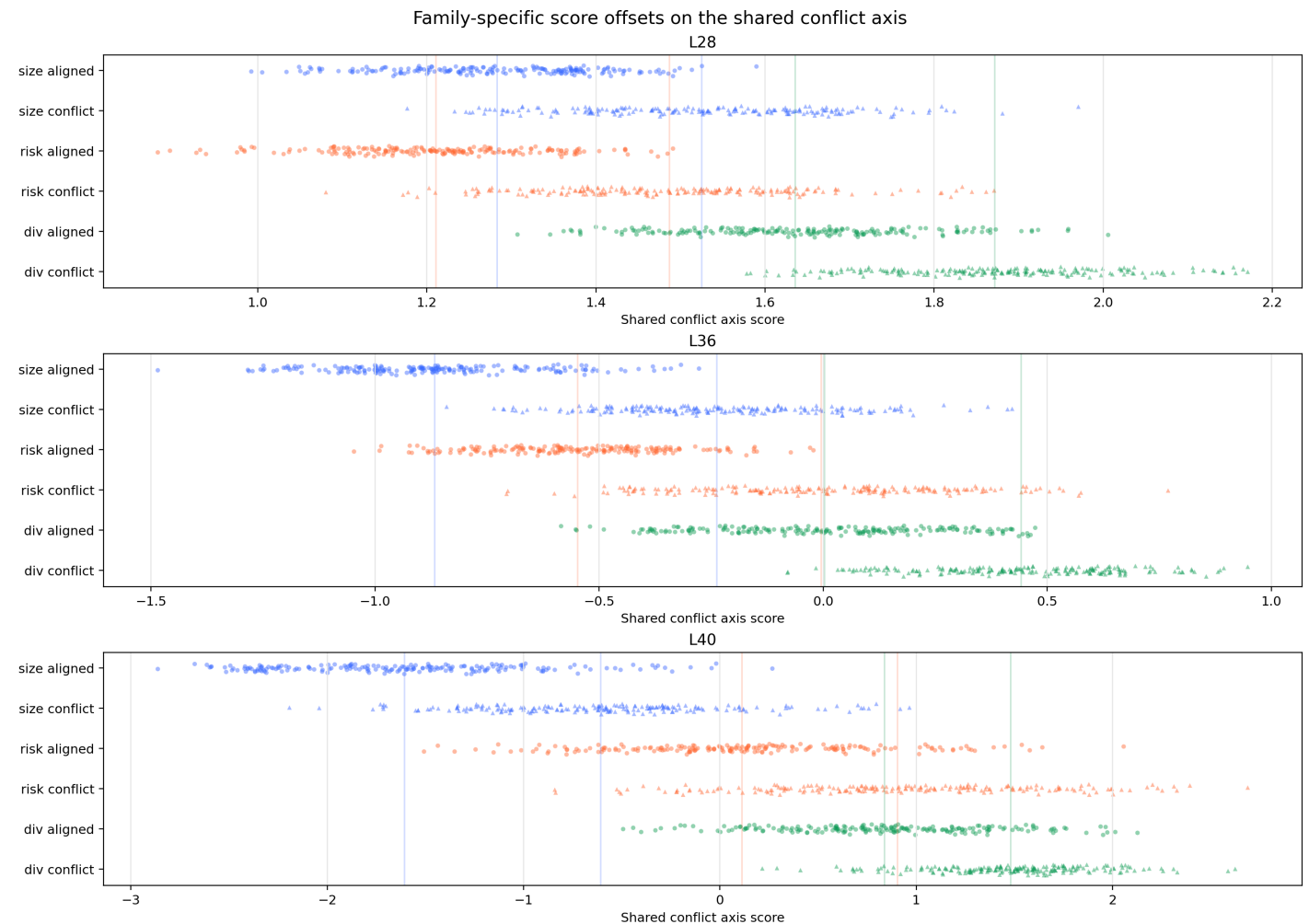


Figure 3: Shared-axis distributions: separation exists, but family baselines are offset.

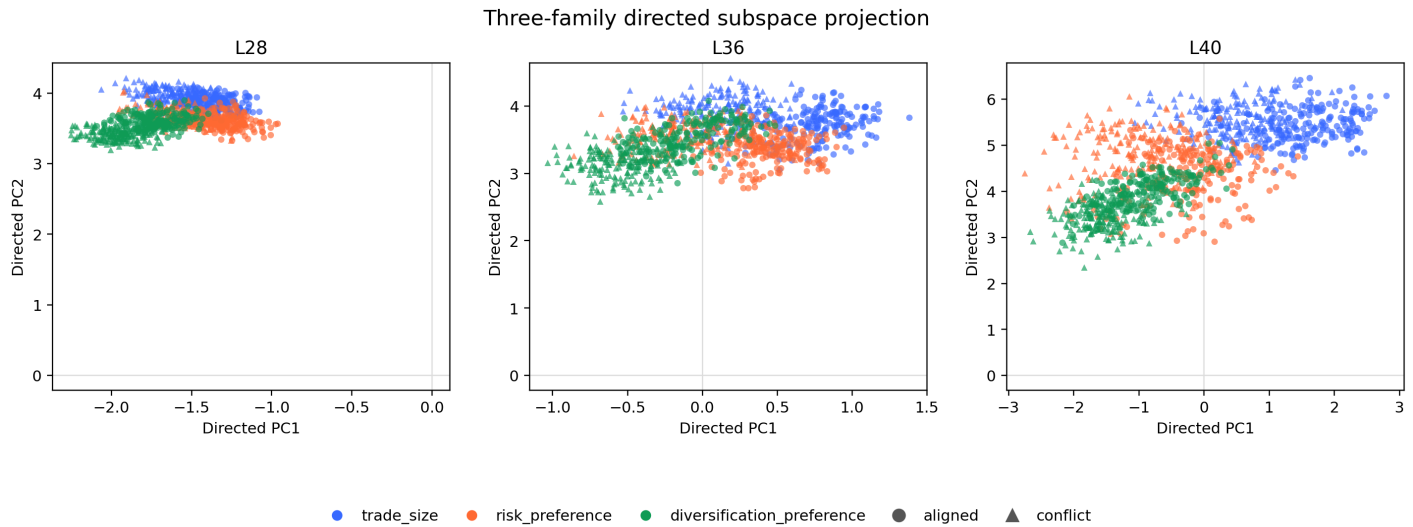


Figure 4: Directed subspace view: related conflict geometry, not one collinear axis.

First Real Transfer Attempt

The first direct transfer pass projected synthetic conflict directions onto full production prompts at coarse global sites. It did not cleanly separate complaint rows from baseline controls.

That failure mattered. It showed that synthetic success does not automatically become production success.

Bridge Program

We then used bridge datasets to separate template mismatch from content and ontology mismatch. The bridge evidence was real but weak: buy-only filtering helped, but the deeper issue was an unresolved ontology and representation mismatch.

Bridge Dataset Counts

Dataset	*Rows*	*Aligned*	*Conflict*
Stage 1a template control	768	384	384
Stage 1b loose adapter	258	168	90
Stage 1b strict adapter	118	81	37
Stage 1b strict buy-only	33	27	6

Source: phase_12/outputs/transfer_bridge/*.json

Phase 13 Real Signal Discovery

Phase 13 asked a simpler question: if we do not train a classifier and do not set thresholds, do fixed synthetic directions produce scalar structure anywhere on real DX Terminal prompts?

L32 Settings-End Cohort Means

Direction	*Anchor*	*Complaint*	*Control*	*Anchor-control*	*Complaint-control*
trade_size	4.425	3.803	3.278	+1.147	+0.526
shared_mean	3.462	3.137	2.760	+0.703	+0.377

Row Reading / Ontology Correction

The most important interpretability move was reading the top and bottom rows. The preregistered root-cause proxy was wrong for the `trade\_size` target. Root-cause labels diagnose why a complaint happened; the probe target is visible current-prefix conflict shape.

Top/Bottom Shape Audit

Direction	*Top action/size*	*Top strategy ignored*	*Bottom action/size*	*Bottom strategy ignored*
trade_size	20/25	5/25	15/25	10/25
shared_mean	20/25	5/25	9/25	16/25

Top trade_size Complaint Types

Type	*Count*
UNWANTED_BUY	10/25
UNWANTED_SELL	6/25
WRONG_SIZE	4/25
Concrete action/size combined	20/25

Claim Boundary

Fixed synthetic directions recover a real production signal at L32 settings_end. `trade\_size` is selective for current-prefix concrete sized-action conflict. `shared\_mean` appears to track broader policy tension, but the shared-family interpretation still needs more audit.

This is not a final detector, not a deployment monitor, and not a causal mechanism claim.

Closing / Next Steps

The useful loop is: real data exposes a messy failure mode; synthetic prompts isolate a clean abstraction; probes find a candidate internal signal; bridge tests expose transfer mismatch; real-data projection finds a narrower shape-specific signal; row reading improves the ontology.